DevOps Infrastructure – Unit 2 Notes (Part 3)

Docker Networking, DNS, Container Communication and Port Mapping

---

1. Docker Networking – Overview

Definition

Docker networking enables communication:

- Between containers
- Between containers and the host system
- Between containers and external networks

Why Networking is Important

- Supports microservices architecture
- Enables service-to-service communication
- Allows exposing applications to users

---

2. Types of Docker Networks

2.1 Bridge Network

Definition

The bridge network is the default network created by Docker for containers running on a single host.

Key Features

- Containers get private IP addresses
- Communication happens within the same host
- Uses a virtual bridge interface

Example

```
docker run -d nginx
```

This container is automatically connected to the default bridge network.

Custom Bridge Network

Better practice is to create a custom bridge network:

```
docker network create my_network

docker run -d --network my_network container1
docker run -d --network my_network container2
```

Now both containers can communicate using names.

2.2 Host Network

Definition

In host networking, the container shares the host's network directly.

Key Features

- No network isolation
- No separate IP address
- Uses host ports directly

Example

```
docker run --network host nginx
```

When to Use

- High-performance applications
- When low latency is required

2.3 Overlay Network

Definition

Overlay networks allow communication between containers across multiple hosts.

Key Features

- Used in distributed systems
- Works with Docker Swarm or Kubernetes
- Enables cross-host communication

Use Case

- Microservices deployed across multiple servers

2.4 None Network

- Disables networking completely
- Container has no external communication

---

3. DNS Inside Docker

Concept

Docker provides an internal DNS system for containers.

How It Works

- Containers can communicate using names instead of IPs
- Docker automatically resolves container names to IP addresses

---

Example

If two containers are in the same network:

container1 can connect to container2 using:
http://container2:port

Advantage

- No need to manually manage IP addresses
- Simplifies service discovery

---

4. Linking Containers

Definition

Linking allows one container to connect to another.

Example

```
docker run -d --name db mysql
docker run -d --link db:webapp app
```

Limitation

- Deprecated feature
- Replaced by custom networks and DNS

---

5. Port Mapping (Host ↔ Container)

Definition

Port mapping connects a container's port to a host machine port.

Syntax

```
docker run -p host_port:container_port image
```

Example

```
docker run -p 8080:80 nginx
```

Explanation

- Port 80 inside container
- Accessible via port 8080 on host

Accessing Application

Open browser:
http://localhost:8080

---

6. Expose vs Publish

EXPOSE (Dockerfile)

- Documentation purpose
- Does not publish port

-p (docker run)

- Actually maps ports
- Makes service accessible

---

7. Summary

- Docker provides multiple network types: bridge, host, overlay
- Bridge is default for single-host communication
- Overlay is used in distributed systems
- Docker has built-in DNS for container communication
- Linking is outdated, use networks instead
- Port mapping connects container services to host

Next part will cover:

- Docker Storage (Volumes vs Bind Mounts)

- Data persistence
- Copy-on-write (revisited in storage context)
- Backing data on host